

Métodos e Atributos Estáticos



O que é um método Estático (Static)

- É o oposto de *métodos de instância*
- São métodos que são da classe e não dos objetos da classe...
- Isto é, são métodos que não faz sentido de serem chamados a partir de objetos porque normalmente não precisam de nada que seja daquele objeto
- Por exemplo
 - método buscarCliente(cli)
 - método listarTodosProdutos()
 - método analisarCreditoCliente (cli)
 - método negativar(cli)
 - método gravarNoBD(produto1)

método save() → instância x estático

```
class Produto {
    std::string nome;
    double preco;

public:
    Produto(std::string n, double p) : nome(n), preco(p) {}

    void save() const {
        // Aqui uma lógica de persistência no banco
        std::cout << "Salvando produto: " << nome << ", R$"
        << preco << std::endl;

        SQL = INSERT INTO PRODUTO VALUES (nome, preco);
    }
};

int main() {
    Produto p("Caneta", 3.50);
    p.save(); // chamada ao método de instância
}
```

```
class Produto {
    std::string nome;
    double preco;

public:
    Produto(std::string n, double p) : nome(n), preco(p) {}

    std::string getNome() const { return nome; }
    double getPreco() const { return preco; }

    static void save(const Produto& p) {
        std::cout << "Salvando produto: " << p.getNome()
        << ", R$" << p.getPreco() << std::endl;

        SQL = INSERT INTO PRODUTO VALUES (p.getNome(), p.getPreco());
    }
};

int main() {
    Produto p("Caneta", 3.50);
    Produto::save(p); // chamada ao método estático
}
```

Atributos Estáticos

- São variáveis que pertencem à classe, não a cada instância da classe
- É como se fosse uma variável global
- Útil para
 - constar instâncias
 - compartilhar estado entre objetos

```
class Contador {
public:
    static int total;

    static void mostrarTotal() {
        std::cout << "Total de objetos: " << total << std::endl;
    }

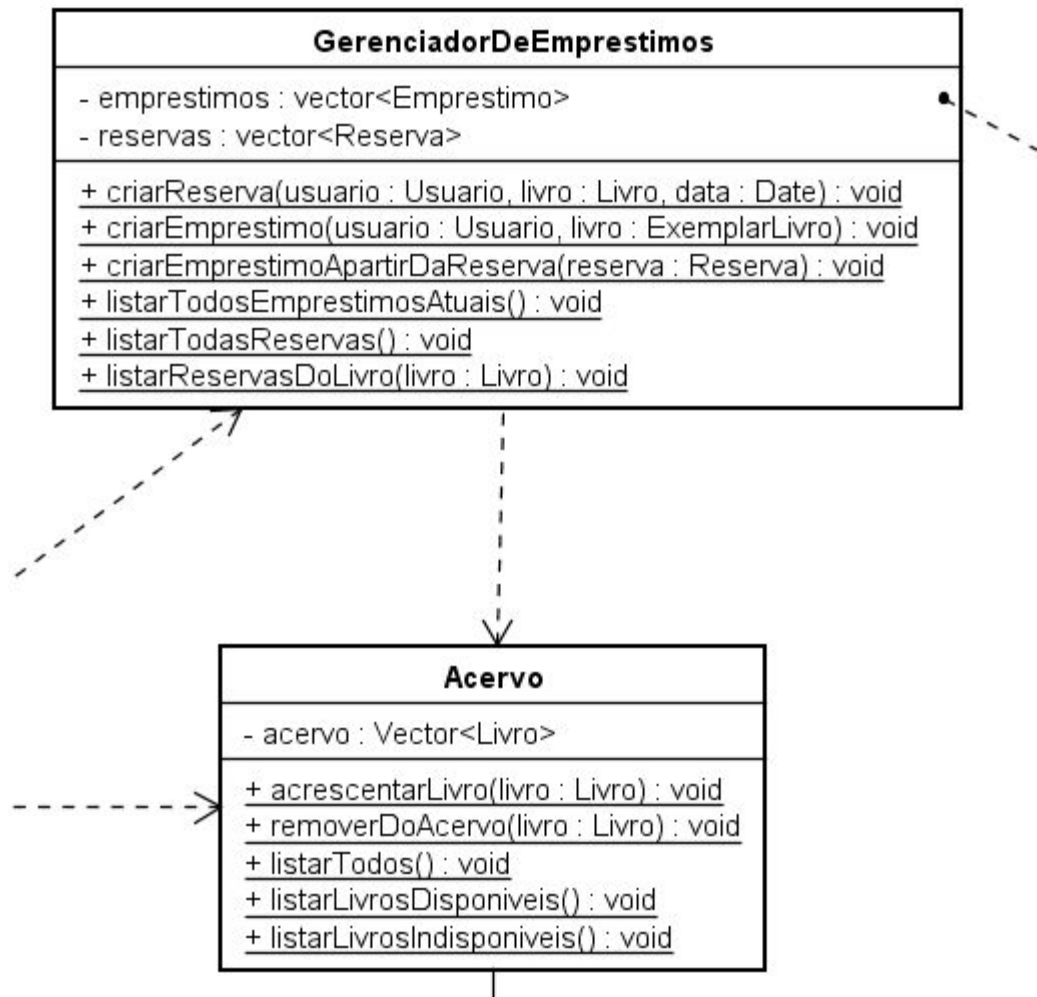
    Contador() { ++total; }
};

int Contador::total = 0;

int main() {
    Contador a, b, c;
    Contador::mostrarTotal(); // Total: 3
}
```

Atributos Estáticos

- Na UML, métodos estáticos são sublinhados



Onde usar no projeto ?

- Métodos estáticos normalmente devem ser colocados em classes que se caracterizam como Gerenciadores, Controladores e outras classes que normalmente manipulam os objetos principais
- As classes que possuem métodos estáticos normalmente não possuem atributos, mas se possuírem normalmente são estáticos também ou públicos



Criando um
menu de opções
no Main

Os próximos slides mostram como criar um menu de opções para o sistema de vocês.

É apenas um exemplo para vocês se basearem, não precisa seguir estritamente o que aparece


```
#include <iostream>
#include <vector>
#include <string>
#include "Funcionario.h"
#include "Projeto.h" // Novo include
using namespace std;

void exibirMenu() {
    cout << "\nMenu:\n";
    cout << "1. Cadastrar novo funcionário\n";
    cout << "2. Alterar funcionário\n";
    cout << "3. Remover funcionário\n";
    cout << "4. Listar todos os funcionários\n";
    cout << "5. Adicionar projeto a um funcionário\n"; // Nova opção
    cout << "6. Sair\n";
    cout << "Escolha uma opcao: ";
}
```

```
void cadastrarFuncionario(vector<Funcionario>& funcionarios) {
    int codigo, idade, cargoOpcao;
    string nome, rua, cidade, estado, cep;

    cout << "Digite o codigo: ";
    cin >> codigo;
    cout << "Digite o nome: ";
    cin.ignore(); // descarta um caracter do buffer anterior de entrada. isso é bom quando teve uma leitura anterior
    getline(cin, nome); //o comando acima é bom para evitar que o getline leia uma linha vazia
    cout << "Digite a idade: ";
    cin >> idade;

    cout << "Digite a rua: ";
    cin.ignore();
    getline(cin, rua);
    cout << "Digite a cidade: ";
    getline(cin, cidade);
    cout << "Digite o estado: ";
    getline(cin, estado);
    cout << "Digite o CEP: ";
    getline(cin, cep);

    Endereco endereco(rua, cidade, estado, cep);

    cout << "Escolha o cargo:\n";
    cout << "0 - CEO\n1 - CTO\n2 - Secretária\n3 - Presidente\n4 - Estagiário\n";
    cout << "Escolha a opcao do cargo (0 a 4): ";
    cin >> cargoOpcao;

    Cargo cargo(static_cast<Cargo::Valor>(cargoOpcao)); //transformando o valor escolhido para um tipo do
```

```
void alterarFuncionario(vector<Funcionario>& funcionarios) {  
    int codigoAlterar;  
    cout << "Digite o codigo do funcionario a ser alterado: ";  
    cin >> codigoAlterar;  
  
    for (auto& f : funcionarios) { //faz o compilador deduzir o tipo do f automaticamente  
        if (f.getCodigo() == codigoAlterar) {  
            string nome;  
            int idade;  
            cout << "Digite o novo nome: ";  
            cin.ignore();  
            getline(cin, nome);  
            f.setNome(nome);  
            cout << "Digite a nova idade: ";  
            cin >> idade;  
            f.setIdade(idade);  
            cout << "Funcionario alterado com sucesso!" << endl;  
            return;  
        }  
    }  
    cout << "Funcionario nao encontrado!" << endl;  
}
```

```
void removerFuncionario(vector<Funcionario>& funcionarios) {
    int codigoRemover;
    cout << "Digite o codigo do funcionario a ser removido: ";
    cin >> codigoRemover;

    for (auto it = funcionarios.begin(); it != funcionarios.end(); ++it) { //usando um interador
        if (it->getCodigo() == codigoRemover) {
            funcionarios.erase(it);
            cout << "Funcionario removido com sucesso!" << endl;
            return;
        }
    }
    cout << "Funcionario nao encontrado!" << endl;
}
```

```
void listarFuncionarios(const vector<Funcionario>& funcionarios) {
    cout << "\nLista de funcionarios:\n";
    for (const auto& f : funcionarios) {
        f.apresentar();
        cout << "-----\n";
    }
}
```

```
void adicionarProjetoFuncionario(vector<Funcionario>& funcionarios) {
    int codigoFuncionario;
    cout << "Digite o codigo do funcionario: ";
    cin >> codigoFuncionario;

    for (auto& f : funcionarios) {
        if (f.getCodigo() == codigoFuncionario) {
            int codigoProjeto;
            string titulo, descricao;
            cout << "Digite o codigo do projeto: ";
            cin >> codigoProjeto;
            cout << "Digite o titulo do projeto: ";
            cin.ignore();
            getline(cin, titulo);
            cout << "Digite a descricao do projeto: ";
            getline(cin, descricao);

            Projeto p(codigoProjeto, titulo);
            f.adicionarProjeto(p);
            cout << "Projeto adicionado com sucesso ao funcionario!\n";
            return;
        }
    }
}
```

```
int main() {  
    vector<Funcionario> funcionarios;  
    int opcao;  
  
    do {  
        exhibirMenu();  
        cin >> opcao;  
  
        switch (opcao) {  
            case 1:  
                cadastrarFuncionario(funcionarios);  
                break;  
            case 2:  
                alterarFuncionario(funcionarios);  
                break;  
            case 3:  
                removerFuncionario(funcionarios);  
                break;  
            case 4:  
                listarFuncionarios(funcionarios);  
                break;  
            case 5:  
                adicionarProjetoFuncionario(funcionarios);  
                break;  
            case 6:  
                cout << "Saindo...\n";  
                break;  
            default:  
                cout << "Opcao invalida! Tente novamente.\n";  
        }  
    }
```